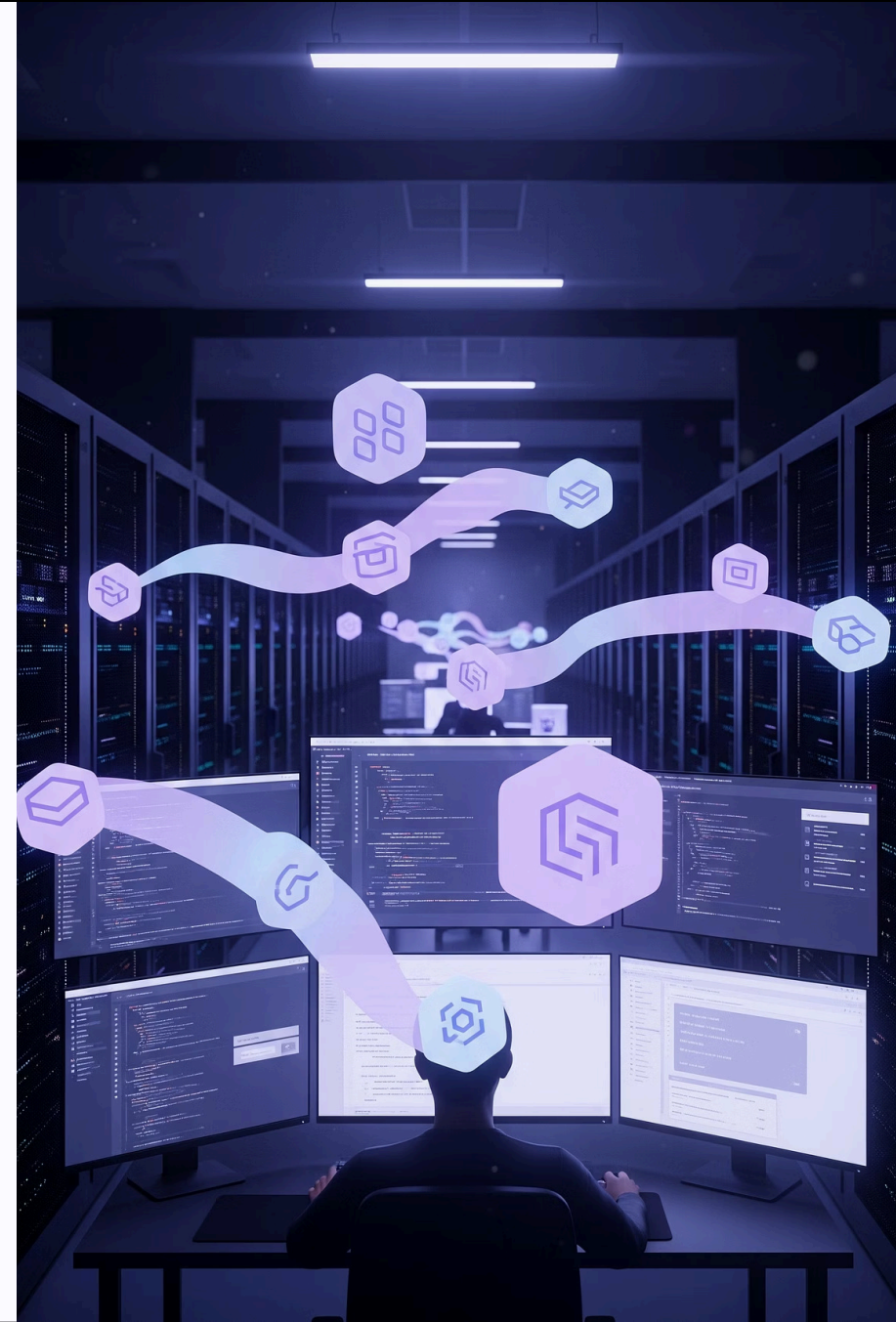


Package Managers: apt, yum, pip

Mastering the Tools for Reproducible and Secure AI Environments

A practical guide for AI infrastructure and ML engineers



Why Package Managers Matter



Efficiency

Package managers automate installation, dependency resolution, and updates—saving hours compared to manual source compilation



Security

Streamlined patch management ensures AI systems stay protected against vulnerabilities



Reproducibility

Essential for creating consistent, version-controlled AI environments across development and production

System-Level vs Python Package Managers



Two Critical Layers

System Layer (apt, yum)

Manages OS libraries, drivers, and core tools

Python Layer (pip, conda)

Manages ML frameworks and Python dependencies

Example: `apt` installs CUDA drivers while `pip` installs PyTorch that leverages those drivers

apt – Debian/Ubuntu Package Manager



Common Commands

```
# Update package index
sudo apt update

# Install packages
sudo apt install nvidia-cuda-toolkit

# Upgrade all packages
sudo apt upgrade
```



Where You'll Find It

- Ubuntu-based cloud VMs
- Most Docker AI containers
- Many research lab environments
- Popular for ML workstations



Key Features

- Uses .deb package format
- Centralized repositories
- Dependency resolution
- Robust security updates

yum & dnf – Red Hat/CentOS Package Managers

Enterprise-Grade Package Management

Basic Usage



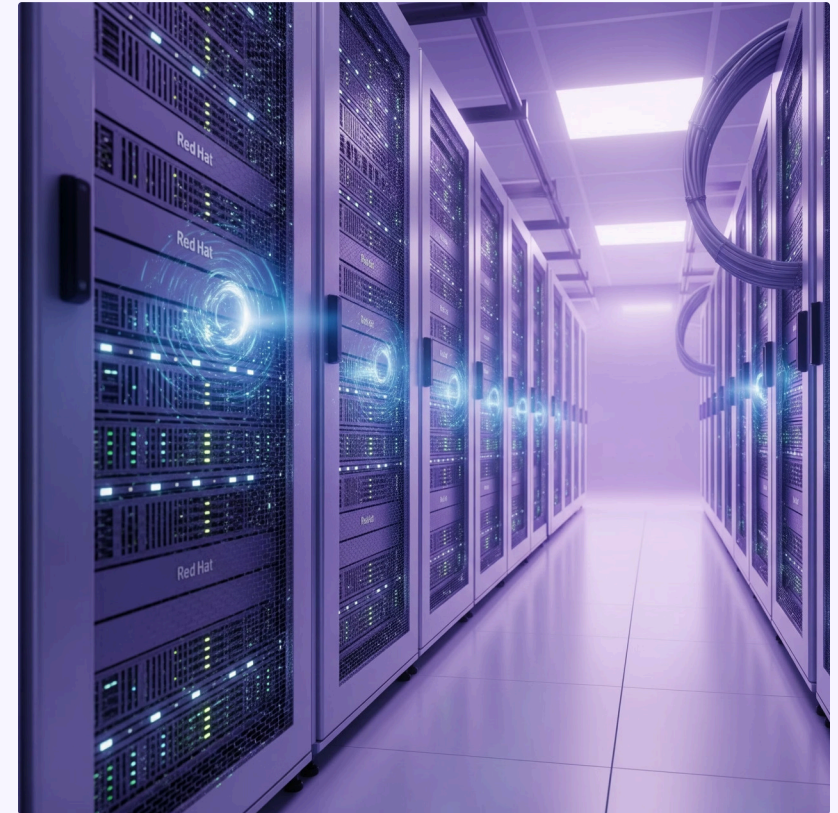
```
# Search for packages  
yum search cuda
```

```
# Install package  
sudo yum install cuda-toolkit
```

Common Environments



- Enterprise GPU servers
- HPC clusters
- Amazon Linux EC2 instances
- Long-term support environments

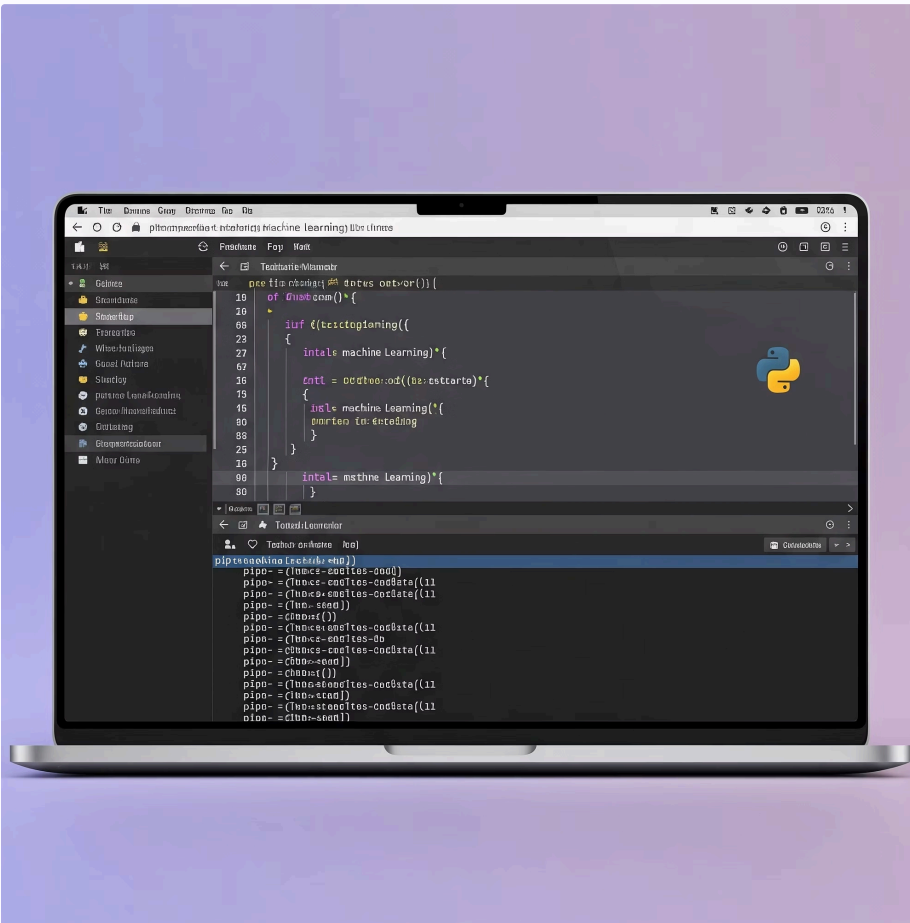


Red Hat Enterprise environments commonly power mission-critical AI infrastructure

pip – Python Package Manager

Essential for ML Engineers

The primary tool for installing Python ML frameworks and their dependencies



Install specific versions

```
pip install torch==2.0.0 torchvision==0.15.1
```

From requirements file

```
pip install -r requirements.txt
```

In virtual environment

```
python -m venv env
```

```
source env/bin/activate
```

```
pip install transformers
```

Integrates with PyPI's vast ecosystem of 400,000+ packages, including all major AI frameworks

Common Pitfalls & Conflicts

⊗ The Dangerous Zone: `sudo pip install`

Never use sudo with pip! This can break system Python and cause major headaches

CUDA Version Mismatches

System CUDA from apt (11.2) ≠
PyTorch's expected CUDA (11.7)

System vs. Virtual Environments

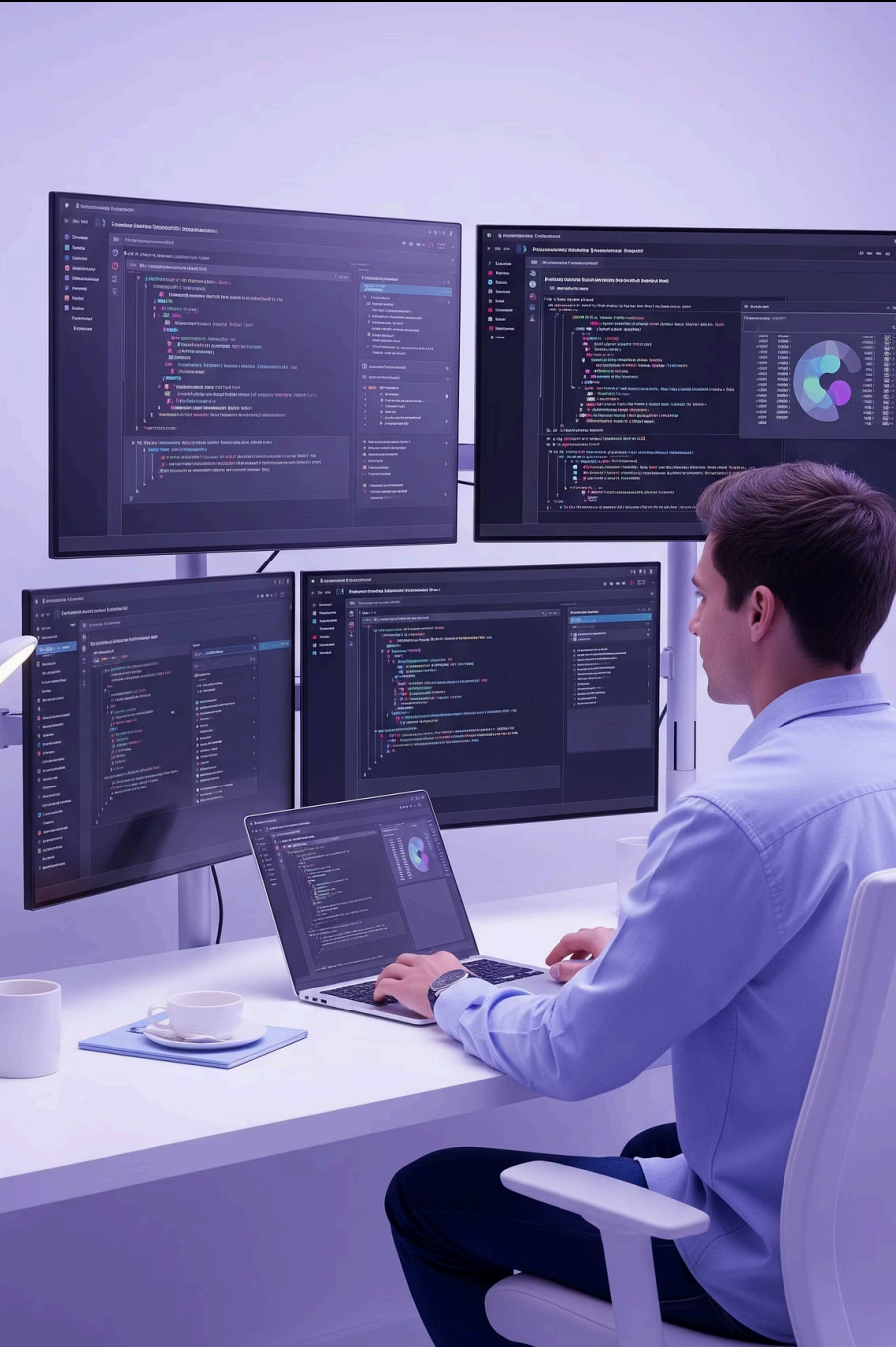
Installing ML packages globally
leads to conflicts between projects

Dependency Resolution Failures

Package A needs numpy 1.20,
Package B needs numpy 1.22

Best Practice

Use virtual environments (venv/conda) for Python packages, apt/yum for system components



Best Practices for AI Engineers

1 Document Everything

Use `requirements.txt` with pinned versions (`==`) for reproducibility

2 Containerize Development

Test installations in containers before applying to production environments

3 Automate Setup

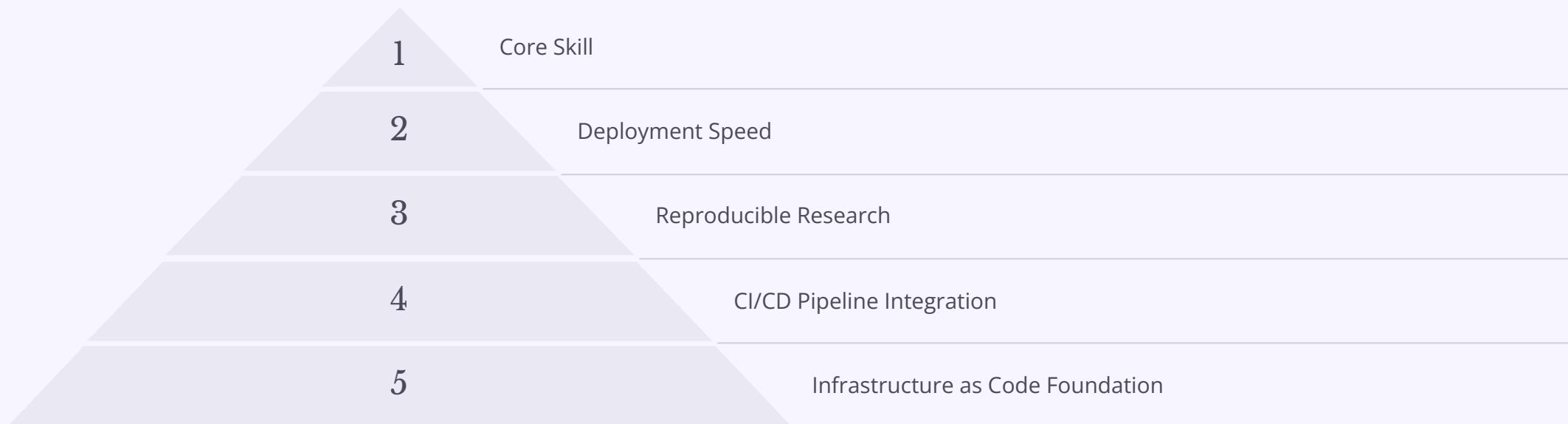
Create setup scripts or Dockerfiles that document the exact installation process

4 Layer Your Approach

First system packages (apt/yum), then Python environment, then ML packages

Why Master Package Managers?

The Bridge Between Infrastructure and ML Innovation



Package management mastery is the **invisible foundation** that enables AI infrastructure engineers to create environments where data scientists can innovate without fighting dependency issues.

Next Steps

Practice creating reproducible environments with Docker and requirements files